Proceedings of Machine Learning Research vol 284:1–22, 2026 20th Conference on Neurosymbolic Learning and Reasoning

Neurosymbolic Program Synthesis for the Abstraction and Reasoning Corpus using Vector Symbolic Algebras

Isaac Joffe* IJOFFE@UWATERLOO.CA and **Chris Eliasmith** CELIASMITH@UWATERLOO.CA
Centre for Theoretical Neuroscience, University of Waterloo

Editors: Alessandra Mileo, Andrea Passerini and Cogan Shimizu

Abstract

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI) is a popular artificial intelligence (AI) benchmark that tests fluid reasoning. Humans solve ARC-AGI with ease, but it is extremely difficult for all but the most advanced AI systems. In this paper, we propose a novel, neurosymbolic ARC-AGI solver with cognitive inspiration from dual process theory and vector symbolic algebra (VSA) research. Our solver structures solutions to ARC-AGI tasks as object-centric programs implemented in a bespoke DSL, and represents these programs using VSAs. Additionally, our solver synthesizes solutions to ARC-AGI tasks by integrating neural System 1 intuitions with procedural System 2 reasoning, and implements both using VSAs. Quantitatively, our solver scores 13.5% on ARC-AGI-1-Train and 3.5% on ARC-AGI-1-Eval, indicating success; qualitatively, our solver is efficient and interpretable, outperforming naive methods by 7× and brute force methods by 235×. Our code is available at: https://github.com/ijoffe/ARC-VSA-2026.

1. Introduction

The Abstraction and Reasoning Corpus for Artificial General Intelligence (ARC-AGI; Chollet, 2019; Chollet et al., 2025c) is a popular but extremely difficult artificial intelligence (AI) benchmark that tests fluid reasoning. ARC-AGI comprises abstract reasoning tasks (e.g., Figures 1 and 2) for which the goal of the solver, human or AI, is simple: discover the hidden rules that govern the symbol transformations in a few demonstrations and apply these rules to solve new queries. ARC-AGI is easy for humans; human performance is estimated to be 85.0%, and all tasks have been solved by multiple humans (Chollet et al., 2025b, 2026). However, ARC-AGI is hard for AI; OpenAI's original GPT-3 model (Brown et al., 2020) scored 0.0% (Chollet et al., 2025b), and the benchmark remains unbeaten despite ample focus and investment over multiple $1,000,000 competitions (Chollet et al., 2024, 2025a).

In this paper, we propose a novel, neurosymbolic ARC-AGI solver with cognitive inspiration from dual process theory and vector symbolic algebra (VSA) research. We extend our past work on applying VSAs to ARC-AGI (Joffe and Eliasmith, 2025) by presenting a novel, VSA-based framework for neurosymbolic program synthesis, which we believe is the first of its kind. Overall, our solver scores 13.5% on ARC-AGI-1-Train and 3.5% on ARC-AGI-1-Eval, indicating strong accuracy; additionally, our best solver proposes on average 7× fewer programs than our naive solver and 235× fewer programs than brute-force search, indicating strong efficiency.

We proceed as follows: first, we introduce ARC-AGI and briefly review dual process theory, VSAs, and program synthesis; second, we describe the workings of our solver; and, third, we demonstrate the various advantages that our solver has over naive methods.

* Corresponding author

© 2026 I. Joffe & C. Eliasmith.

2. Background

Fundamentally, we aim to develop a cognitively inspired ARC-AGI solver. To this end, we appeal to principles that originate from cognitive research spanning psychology (e.g., dual process theory) to neuroscience (e.g., VSAs) and work at multiple levels of abstraction.

ARC-AGI ARC-AGI defines intelligence as *skill-acquisition efficiency*, a system's ability to rapidly develop new skills to solve new problems (Chollet, 2019), and aligns with this paradigm by forcing solvers to learn arbitrary transformations on-the-fly from few examples.

The benchmark is a collection of abstract reasoning *tasks* (e.g., Figures 1 and 2). Each task comprises a *training* component, a few *demonstrations* of *input* grids mapped to *output* grids, and a *testing* component, usually one lone input grid *query*. The solver's goal is to infer the latent rules that underlie the demonstrations and apply them to the queries.

ARC-AGI is difficult because it is generative, invokes abstract concepts, and enforces task uniqueness. Despite these challenges, ARC-AGI is based solely on four core knowledge priors: *objectness*, *goal-directedness*, *numbers*, and *geometry*. Solvers that understand these four ideas can theoretically solve all ARC-AGI tasks; specialized knowledge is unnecessary.

Dual Process Theory In psychology, the mind is often understood as two fictitious agents, System 1 and System 2 (Wason and Evans, 1974; Kahneman, 2011). System 1 performs fast, automatic tasks using heuristic intuitions; System 2 performs slow, intentional tasks using careful reasoning. Likewise, AI is dominated by two paradigms, connectionism and symbolicism. Connectionist models, usually implemented with neural networks (NNs; LeCun et al., 2015; Schmidhuber, 2015), excel at System 1 tasks; symbolic models, often implemented with rules, excel at System 2 tasks. Because of the complementary nature of these types of intelligence, it is important to model both (Bengio et al., 2021).

Vector Symbolic Algebras VSAs[1] (for a comprehensive review, see Kleyko et al., 2022, 2023) are a family of algebras that represent symbols as high, N-dimensional vectors and define operations over these neurosymbolic vectors. In this paper, we exclusively use Holographic Reduced Representations (HRRs; Plate, 1991, 2003), one particular VSA.

HRRs define three core operations: *similarity*, *bundling*, and *binding*. Similarity, denoted by $\cdot$ and implemented as the dot product, combines two vectors into a scalar that captures the semantic likeness of the symbols represented by the input vectors. Bundling, denoted by $+$ and implemented as element-wise addition, combines two vectors into another that has partial similarity to both input vectors. Binding, denoted by $\circledast$ and implemented as circular convolution, combines two vectors into another that has low similarity to both input vectors. These operations can be used to compute useful cognitive functions (Eliasmith et al., 2012).

Spatial Semantic Pointers (SSPs; Komer et al., 2019) extend HRRs to represent continuous spaces while accurately modelling neurons observed in the brain (Dumont and Eliasmith, 2020). An SSP encoding defines a mapping $\phi : \mathbb{R}^D \to \mathbb{R}^N$ from a low-, D-dimensional feature space to the VSA vector space based on *fractional binding* as follows:

$$\phi(\boldsymbol{x}) = \mathcal{F}^{-1}\{e^{i\Theta\frac{\boldsymbol{x}}{l}}\} \tag{1}$$

where $\mathcal{F}\{\cdot\}$ is the discrete Fourier transform, i is the imaginary unit, $\Theta \in \mathbb{R}^{N\times D}$ is the phase matrix that defines the encoding, and $l \in \mathbb{R}$ is a length-scale hyperparameter. SSPs

1. Also known as vector symbolic architectures or hyperdimensional computing.

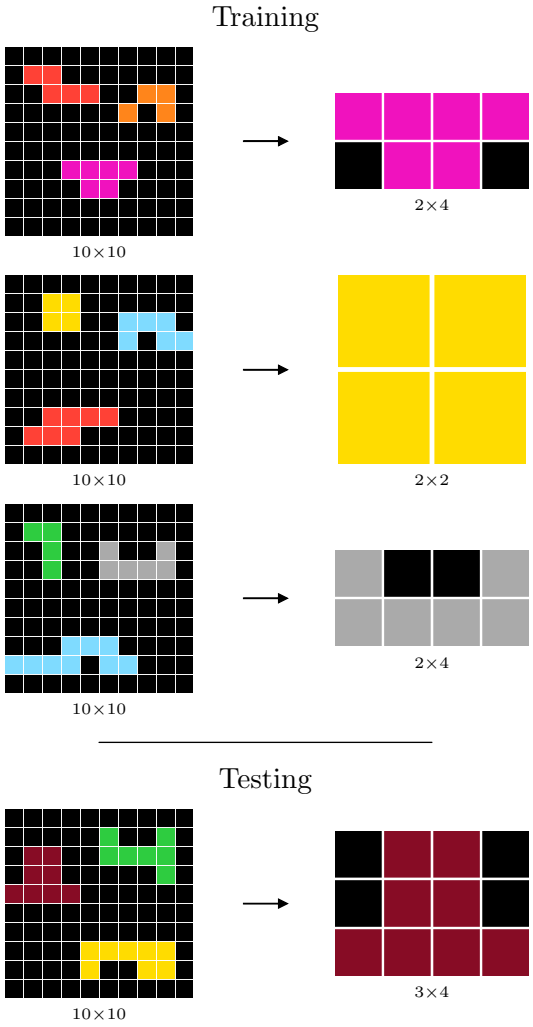


Figure 1: ARC-AGI task `72ca375d`.

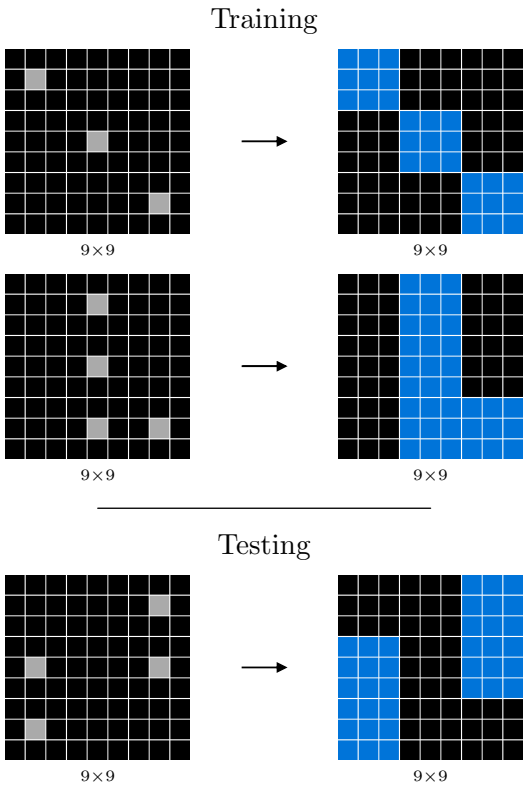


Figure 2: ARC-AGI task `ce22a75a`.

1. $\mathcal{O}_\mathrm{I} \leftarrow$ TwoConnectedHypothesis(G_I)
2. $\mathcal{O}_\mathrm{O} \leftarrow$ Crop(Filter($\mathcal{O}_\mathrm{I}$, HasHorizontalSymmetry($\mathcal{O}_\mathrm{I}$)))
3. $G_\mathrm{O} \leftarrow$ DecodeObjects($\mathcal{O}_\mathrm{O}$)

Figure 3: Our solver's solution program for ARC-AGI task `72ca375d`.

1. $\mathcal{O}_\mathrm{I} \leftarrow$ ColourHypothesis(G_I)
2. $\mathcal{O}_\mathrm{O} \leftarrow$ Encircle(Recolour($\mathcal{O}_\mathrm{I}$, ColourConstant(Blue)), ColourConstant(Blue))
3. $G_\mathrm{O} \leftarrow$ DecodeObjects($\mathcal{O}_\mathrm{O}$)

Figure 4: Our solver's solution program for ARC-AGI task `ce22a75a`.

can be combined with symbolic HRRs to represent complex scenes, such as ARC-AGI grids.

VSAs are useful for reasoning because they can encode hierarchical information with compositional structure in a unified substrate. For example, VSAs have been used to model how humans solve (Rasmussen and Eliasmith, 2011) Raven's Progressive Matrices (RPMs; Raven, 1936), the long-standing intelligence test by which ARC-AGI was inspired. By infusing syntactic structure into distributed representations, VSAs bridge connectionism and symbolicism and offer a principled method for neurosymbolic AI (Eliasmith, 2013).

Program Synthesis Neurosymbolic program synthesis (for a comprehensive review, see Chaudhuri et al., 2021) augments conventional program synthesis (e.g., Gulwani, 2011) with neural guidance. In this paper, we study *neural program synthesis* (e.g., Menon et al., 2013) rather than *neural program induction* (e.g., Devlin et al., 2017) because we work with explicit neurosymbolic programs. We take primary inspiration from DeepCoder (Balog et al., 2017) as well as program sketching (Murali et al., 2018; Nye et al., 2019) and execution guidance (Chen et al., 2019; Ellis et al., 2019), but uniquely implement our methods using VSAs.

3. Methods

We believe that humans solve ARC-AGI tasks using object-centric programs synthesized with contributions from both System 1 and System 2 (Johnson et al., 2021; Acquaviva et al., 2022; LeGris et al., 2025). System 1 interprets the task's demonstrations in terms of objects and proposes ideas about the task's hidden patterns; System 2 methodically devises an explicit program based on those objects that comprehensively explains the observed patterns. We posit that artificial ARC-AGI solvers should also work by object-centric program synthesis realized using the interplay of System 1 intuition and System 2 reasoning.

3.1. Solution Structure

Our solver structures solutions as object-centric programs.

3.1.1. Objects

Humans reason about ARC-AGI tasks with mental *objects* rather than with individual pixels or whole grids (Johnson et al., 2021). An object is a group of pixels transformed cohesively. Because transformations are necessarily different in every task, what makes a group of pixels constitute an object is task-dependent. Therefore, an explicit object hypothesis (i.e., rules that define whether an arbitrary group of pixels forms an object) is part of all programs.

We store each object as the foreground mask of its pixels. Pixels not involved in any transformation are not part of any object; instead, they are treated as background pixels.

3.1.2. Programs

Humans solve ARC-AGI tasks with explicit mental *programs* (Acquaviva et al., 2022). A program is a deterministic sequence of steps that can be executed to map any valid input grid to a valid output grid. For a particular task, a program is *consistent* if it solves all the demonstrations and *correct* if it is consistent and solves all the queries. In our view, solving an ARC-AGI task therefore amounts to synthesizing a correct program for that task.

Importantly, the programs that humans use to solve ARC-AGI tasks invoke abstract concepts (e.g., motion, selection) often grounded in metaphors to past experience (LeGris et al., 2025). Accordingly, we implement programs in a carefully designed DSL that provides strong inductive biases by capturing ideas that commonly appear in ARC-AGI while remaining as general as possible. Our DSL's primitives each model one irreducible operation or construct that humans use, supporting natural compositionality. Fortunately, ARC-AGI is amenable to such a DSL because of its discrete nature and narrow grid-based domain.

Architecture Programs in our DSL (e.g., Figures 3 and 4) have a three-step architecture.

First, the input grid $G_\mathrm{I} \in \mathbb{G}$ is parsed into a set of input objects $\mathcal{O}_\mathrm{I}$. Our DSL has five object hypothesis primitives (Table 1) that capture common, useful ways to perceive objects. For example, our DSL has a GridHypothesis primitive that parses grids into single objects containing all pixels and a TwoConnectedHypothesis primitive that parses grids into objects whose pixels are all the same colour and are within two degrees of spatial proximity.

Second, this set of input objects $\mathcal{O}_\mathrm{I}$ is transformed into a set of output objects $\mathcal{O}_\mathrm{O}$. Transformations are compositions of primitives, together with optional constant parameters, that adhere to our DSL's type requirements. Our DSL has four types (Table 2) that store different data required to solve ARC-AGI tasks. Specifically, our DSL has a fundamental Object type, which is at the core of all program logic, as well as auxiliary Colour, Number, and Boolean types. Tasks are characterized by how objects are transformed, and objects can be transformed in many ways, so the majority of our DSL's primitives serve this step. Our DSL has 34 transformation primitives (Tables 3 and 4) that each process sets of data of particular types. For example, our DSL has primitives that re-permute (e.g., VerticalFlip), augment (e.g., Encircle), combine (e.g., PixelwiseAnd), select (e.g., Filter), and retrieve information about (e.g., HasHorizontalSymmetry) objects.

Third, this set of output objects $\mathcal{O}_\mathrm{O}$ is recombined into an output grid $G_\mathrm{O} \in \mathbb{G}$. Objects are merged by overlaying their individual foreground masks onto a common grid; remaining background pixels take the overall grid background colour, which is assumed to be black.

Representation We represent primitives as symbols (i.e., each primitive q has a representative vector $\mathtt{Q} \in \mathbb{R}^N$). Primitive representations can be assigned randomly, intelligently engineered by humans, learned from data, or otherwise generated in a structured manner.

We represent programs as bundles of the representations of their constituent primitives (i.e., each program p with primitives $\mathcal{Q}_p$ has a representative vector $\mathtt{P} = \sum_{q \in \mathcal{Q}_p} \mathtt{Q} \in \mathbb{R}^N$). Importantly, the similarity between the representations $\mathtt{Q}$ and $\mathtt{P}$ of a primitive q and a program p approximately indicates whether q is in p. Note that, because of the commutativity of bundling, program representations are not unique; VSAs can support unique program representations, but this is out of the scope of this paper.

3.2. Solution Synthesis

Our solver synthesizes solutions by modelling System 1 intuition and System 2 reasoning.

3.2.1. System 1

We model System 1 by heuristically scoring candidate programs. Note that the scores produced by this System 1 model need not—and, in fact, should not—be perfect. It is not

the role of System 1 to produce the exact solution program; instead, the role of System 1 is merely to bias the subsequent System 2 search process towards plausible programs.

Naive Neural Scoring Inspired by DeepCoder (Balog et al., 2017), we use an NN f_{Naive}, parametrized by θ_{Naive}, to predict the presence or absence of each individual primitive in a task's solution program. The input to f_{Naive} is a perceptual representation of a demonstration $(\mathtt{G}_{\text{I}}, \mathtt{G}_{\text{O}}) \in \mathbb{R}^{2N}$ generated using a VSA-based encoding scheme $G \mapsto \mathtt{G}$. The output from f_{Naive} is a vector of K Bernoulli probabilities $\boldsymbol{q}_{\text{Naive}} \in \mathbb{R}^K$ where each element $\boldsymbol{q}_{\text{Naive}}(q)$ indicates the likelihood that the primitive q occurs in the program p that maps G_{I} to G_{O}. This NN, formally $f_{\text{Naive}}(\mathtt{G}_{\text{I}}, \mathtt{G}_{\text{O}}; \theta_{\text{Naive}})$, is implemented as a simple multi-layer perceptron with logistic output activations and trained by mini-batch gradient descent on binary cross-entropy loss. We use f_{Naive} to naively score candidate programs with the product of the predicted probabilities of their constituent primitives (i.e., the score of a program p with primitives $\mathcal{Q}_p$ is $\prod_{q \in \mathcal{Q}_p} \boldsymbol{q}_{\text{Naive}}(q)$). Because each $\boldsymbol{q}_{\text{Naive}}(q)$ is a probability, additional primitives cannot increase a program's score, incorporating implicit regularization.

To train f_{Naive}, we generate synthetic data as follows: first, we randomly sample a program in our DSL; second, we randomly generate an input grid conducive to the sampled program; and, third, we apply this program to the input grid to produce an output grid. Crucially, we only sample programs with one Object-producing primitive. This restriction keeps generation tractable and enables us to directly test our solver's out-of-distribution, compositional generalization using complex tasks that require deep programs.

Although surprisingly effective, naive scoring is simplistic and ignores regularities in the structure of programs that solve ARC-AGI tasks. For example, primitives tend not to repeat (e.g., the Crop primitive usually need only be used once), and conceptually similar primitives tend not to co-occur (e.g., the Rotate90 and Rotate270 primitives accomplish the same high-level goal of object re-permutation, but accomplish nothing when composed). We use the aforementioned VSA representations of primitives and programs together with a novel program scoring method to exploit these regularities, improving reasoning efficiency.

VSA-Based Scoring We reuse f_{Naive} to indirectly predict the representation of a task's solution program with f_{VSA}. Specifically, we use f_{Naive} as normal (i.e., with a representation of a demonstration as input and a vector of probabilities as output), but project the predicted probabilities of the primitives $\boldsymbol{q}_{\text{Naive}} \in \mathbb{R}^K$ into a predicted representation of a program $\mathtt{R} \in \mathbb{R}^N$. We compute this projection by bundling the representation $\mathtt{Q}$ of each primitive q weighted by its corresponding predicted probability $\boldsymbol{q}_{\text{Naive}}(q)$ (i.e., $\mathtt{R} = \mathbf{Q}^\intercal \boldsymbol{q}_{\text{Naive}}$ where the rows of $\mathbf{Q} \in \mathbb{R}^{K \times N}$ are the representations of the primitives). Note that the similarities between $\mathtt{R}$ and the representations of primitives whose representations have mutual similarity will be higher than those primitives' predicted probabilities (i.e., $\mathtt{R} \cdot \mathtt{Q} > \boldsymbol{q}_{\text{Naive}}(q)$).

We use f_{VSA} to score candidate programs using a recursive algorithm. The algorithm searches for programs that produce output of type τ based on a representation $\mathtt{R}$; initially, $\mathtt{R}$ is as predicted by f_{VSA} and τ is Object because all programs must produce objects. The algorithm proceeds by computing the similarity between $\mathtt{R}$ and the representation $\mathtt{Q}_\tau$ of each valid (i.e., τ-producing) primitive q_τ to approximate the probability that each q_τ should be used. The algorithm then, in a beam search approach, selects the top k primitives and recursively invokes itself to search for sub-programs that produce each input to each selected primitive. Crucially, recursive calls of the algorithm use a new representation $\mathtt{R}_{\text{New}}$ obtained

by *unbundling* (i.e., subtracting) the representation $\mathtt{Q}_{\text{Selected}}$ of the selected primitive q_{Selected} from $\mathtt{R}$ (i.e., $\mathtt{R}_{\text{New}} = \mathtt{R} - \mathtt{Q}_{\text{Selected}}$), and a new target type τ_{New} that matches the input type of q_{Selected}. This allows the algorithm to continue its search for deeper programs without the portions of $\mathtt{R}$ that have already been explained. Finally, the algorithm scores completed programs with the product of the similarities between the representations of each of that program's constituent primitives and $\mathtt{R}$ at the time that each primitive was selected. With this algorithm, primitive representations are important because unbundling decreases the future similarities of not only the selected primitive, but also other primitives with similar representations. For the non-redundant structure of programs to be exploited, the similarity between the representations of two primitives must therefore correspond to their practical similarity (i.e., approximate interchangeability or perceptual indistinguishability).

Perceptual Representations Different perceptual features of a task are necessary to accurately and robustly suggest the different primitives that may underlie that task's transformations. Therefore, engineering the VSA-based encoding scheme—which may combine discrete representations (e.g., for colour) and continuous representations (e.g., for position) via Equation (1) using HRRs—that generates the perceptual representations used as input to the NNs is a powerful way to inject inductive biases into our solver. We propose three such schemes: first, a scheme that encodes the position of all background (i.e., black) and foreground (i.e., non-black) pixels via Equation (3); second, a scheme that encodes the position and colour of all foreground pixels via Equation (4); and, third, a scheme that encodes the number of foreground pixels of each colour via Equation (5). Each of these schemes helps the NNs to detect certain primitives, so we ensemble NNs trained on data encoded by each to leverage their complementary strengths. We can then score candidate programs using an NN ensemble by either scoring the aggregate of the NN predictions or aggregating the scores produced by the NN predictions individually.

3.2.2. System 2

We model System 2 by executing and assessing candidate programs.

Enumeration Again inspired by DeepCoder (Balog et al., 2017), we use enumeration to search through our DSL. We enumerate programs as follows: first, we score a subset of all possible programs; second, we rank these candidate programs in order of decreasing score; and, third, we attempt these ranked programs one-by-one until a consistent one is found.

Although surprisingly effective, naive enumeration is simplistic; executing programs independently and in an order based solely on initial intuitions does not accurately model the System 2 reasoning process used by humans. We use the aforementioned VSA representations of primitives and programs together with a novel program search method to model the evolutionary component of search, improving reasoning efficiency.

Evolution We believe that when solving ARC-AGI tasks, humans use feedback from the outputs of previously attempted programs to guide the selection of future programs to attempt. If a program transforms the input grids into predicted grids like the correct output grids, then that program is partially successful and its underlying ideas warrant further development. Otherwise, that program is unsuccessful and its underlying ideas should be abandoned. Importantly, the perceptual features of grids that determine how similar or

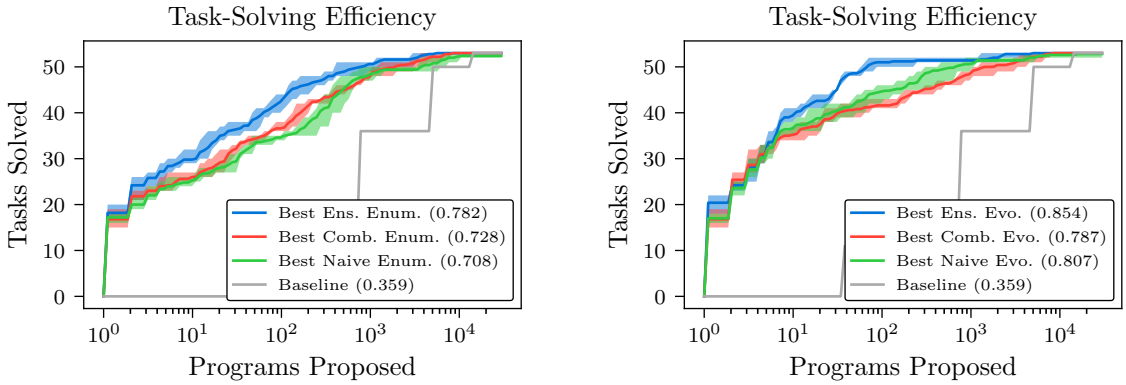


Figure 5: Our solver performs better when it uses an NN ensemble for scoring, regardless of whether it uses enumeration (left) or evolution (right) for search. Lines are means and shaded regions cover the minimum and maximum number of tasks solved across five trials; numbers are normalized areas under the mean curves.

different they appear are the same ones represented with our VSA-based encoding schemes. Therefore, the basic logic that dictates the progression of search can be implemented by computing a VSA-based improvement score $\mathtt{G}_\mathrm{P} \cdot \mathtt{G}_\mathrm{O} - \frac{\mathtt{G}_\mathrm{P} \cdot \mathtt{G}_\mathrm{I} + \mathtt{G}_\mathrm{O} \cdot \mathtt{G}_\mathrm{I}}{2}$.

We also believe that when solving ARC-AGI tasks, humans iterate through high-level ideas rather than through individual, precisely specified programs. Importantly, the essence of a program is encoded in its VSA representation. Therefore, programs that implement the same high-level idea can be grouped by computing a VSA-based similarity score.

We use these techniques to search through programs using an evolutionary algorithm. At each iteration, the algorithm generates an active set $\mathcal{A}$ of programs by selecting the highest-scored remaining program and including additional programs if the differences between their score and the already included programs' scores are minimal and the similarities between their representation and the already included programs' representations are maximal. All the programs in $\mathcal{A}$ are then attempted together. If any of the programs in $\mathcal{A}$ is consistent, then synthesis may terminate. Otherwise, if any of the programs in $\mathcal{A}$ is partially successful (i.e., any program in $\mathcal{A}$ predicts grids with a positive improvement score), then the high-level idea embodied by $\mathcal{A}$ is considered further because it may contribute to solving the task. Consequently, the next active set is conditioned on the most successful partial program (i.e., the program in $\mathcal{A}$ whose predicted grids had the highest improvement score) by re-scoring candidate programs by their ability to solve the sub-task of transforming the predicted grids into the output grids. Using the NNs with this sub-task's demonstrations reveals the primitives required to complete the rest of the task's transformation because the perceptual cues that indicated the use of the partial primitives disappear after the partial program is applied; to encourage re-use of the partial primitives, we fix all their predicted probabilities to 1. Until all plausible conditioned programs have been exhausted without further improvement, future active sets are generated using the conditioned program scores.

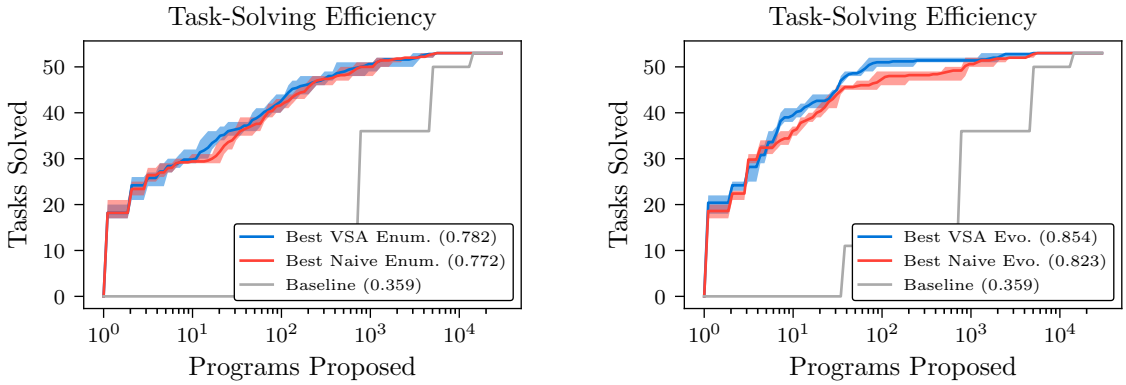


Figure 6: Our solver performs slightly better when it uses VSA representations for scoring, regardless of whether it uses enumeration (left) or evolution (right) for search.

Otherwise, if all the programs in $\mathcal{A}$ were unsuccessful (i.e., each program in $\mathcal{A}$ predicts grids with a negative improvement score), then the high-level idea embodied by $\mathcal{A}$ is abandoned because it is unlikely to contribute to solving the task. Consequently, the next active set is generated as normal. Importantly, this algorithm ensures that feedback from the attempts of all related programs is consolidated before a decision about search progression is made, minimizing the risk of a disproportionate response to a premature signal of partial success or failure. Additionally, decomposing tasks into sub-tasks easily solved individually allows our solver to escape its training distribution and generalize to novel compositions of primitives.

4. Results

We evaluate our solver on a subset of 53 ARC-AGI-1-Train tasks that are solvable in our DSL using two or fewer object-producing primitives. We assess performance with *task-solving efficiency* rather than pure accuracy because, in principle, brute-force search can achieve perfect accuracy. Specifically, we count the number of programs proposed before a correct one is found and measure the area under the cumulative task-solving efficiency curve. As a baseline, we enumerate programs in order of increasing primitive count.

Overall, our solver scores 13.5% on ARC-AGI-1-Train and 3.5% on ARC-AGI-1-Eval.

VSA-Based Perceptual Representations Scoring programs using ensembled NNs that accept perceptual representations encoded by different VSA-based schemes as input is better than using a single NN that accepts the naive scheme given in Equation (2), a combination (i.e., bundle) of those schemes, or any of those schemes individually (Figure 5).

VSA-Based Program Representations Scoring programs using human-engineered VSA-based program representations is slightly better than scoring programs naively (Figure 6).

VSA-Powered Evolution Searching through programs using our VSA-powered evolutionary algorithm is much better than using enumeration algorithms (Figure 7).

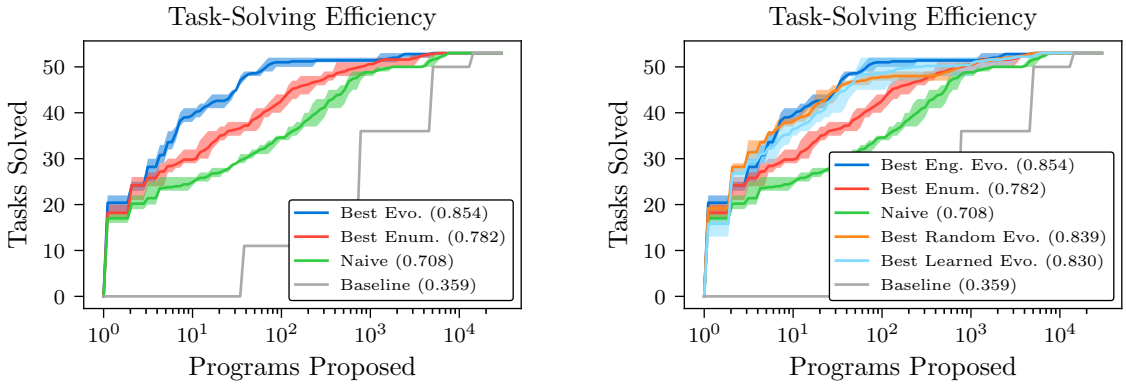


Figure 7: Our solver performs significantly better when it uses VSA-powered evolution for search (left), even with random or learned primitive representations (right).

Strengths First, our solver is efficient. After proposing only 10 programs, our best solver can solve a median of 40 ARC-AGI tasks, while our naive solver can solve a median of only 25; after proposing 100 programs, these figures are 51 and 35. By median number of proposed programs per task, our best solver is 7× more efficient than our naive solver and 235× more efficient than brute-force search; by mean number of proposed programs per task, these figures are 11× and 59×. Second, our solver is interpretable. Solutions are structured as explicit programs and synthesized using opaque decision-making processes, and our evolutionary algorithm proposes a qualitatively reasonable sequence of programs. Third, we believe our solver is general. Our methods for program scoring and program search may be applicable to other domains in which the program inputs and outputs can be represented using VSAs. Fourth, we believe our solver is scalable, albeit manually. Although our DSL is frozen, our solver may be endowed with the additional inductive biases required to solve novel tasks by creating new primitives, generating new synthetic training data, and, if necessary, engineering new VSA-based encoding schemes.

Limitations Raw scores are not state-of-the-art because our DSL has insufficient coverage of the concepts in ARC-AGI. Also, our evolutionary algorithm assumes that every partial program produces a positive improvement score; if this is violated, performance will falter.

5. Conclusion

In this paper, we presented a novel, neurosymbolic, cognitively inspired ARC-AGI solver. Our solver's solutions were structured as VSA-representable programs in an object-centric DSL, and were synthesized using dynamic System 2 reasoning guided by VSA-based System 1 intuitions. Quantitatively, our solver scored 13.5% on ARC-AGI-1-Train and 3.5% on ARC-AGI-1-Eval, indicating success; qualitatively, our solver was efficient and interpretable, outperforming naive methods by 7× and brute force by 235×. Overall, our unique approach bridges cognitive modelling, neurosymbolic AI, and program synthesis using VSAs.

Acknowledgments

The authors would like to thank Varun Dhanraj and Marvin Pafla for discussions that helped improve this paper. This work was supported by CFI (52479-10006) and OIT (35768) infrastructure funding as well as the Canada Research Chairs program, NSERC Discovery grant 261453, AFOSR grant FA9550-17-1-0644 and NSERC CGS-M graduate funding.

References

Sam Acquaviva, Yewen Pu, Marta Kryven, Theodoros Sechopoulos, Catherine Wong, Gabrielle Ecanow, Maxwell Nye, Michael Tessler, and Josh Tenenbaum. Communicating natural programs to humans and machines. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 3731–3743. Curran Associates, Inc., 2022.

Matej Balog, Alexander L. Gaunt, Marc Brockschmidt, Sebastian Nowozin, and Daniel Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations*, 2017.

Yoshua Bengio, Yann LeCun, and Geoffrey E. Hinton. Deep learning for AI. *Communications of the ACM*, 64(7):58–65, 2021.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

Swarat Chaudhuri, Kevin Ellis, Oleksandr Polozov, Rishabh Singh, Armando Solar-Lezama, and Yisong Yue. Neurosymbolic programming. *Foundations and Trends in Programming Languages*, 7(3):158–243, 2021.

Xinyun Chen, Chang Liu, and Dawn Song. Execution-guided neural program synthesis. In *International Conference on Learning Representations*, 2019.

Francois Chollet. On the measure of intelligence. https://arxiv.org/abs/1911.01547, 2019.

Francois Chollet, Mike Knoop, Bryan Landers, Greg Kamradt, Hansueli Jud, Walter Reade, and Addison Howard. ARC Prize 2024. https://kaggle.com/competitions/arc-prize-2024, 2024. Kaggle.

Francois Chollet, Mike Knoop, Greg Kamradt, Walter Reade, and Addison Howard. ARC Prize 2025. https://kaggle.com/competitions/arc-prize-2025, 2025a. Kaggle.

Francois Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2024: Technical report. https://arxiv.org/abs/2412.04604, 2025b.

Francois Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. https://arxiv.org/abs/2505.11831, 2025c.

François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC Prize 2025: Technical report. https://arxiv.org/abs/2601.10904, 2026.

Jacob Devlin, Jonathan Uesato, Surya Bhupatiraju, Rishabh Singh, Abdel-rahman Mohamed, and Pushmeet Kohli. RobustFill: Neural program learning under noisy I/O. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML'17, pages 990–998. JMLR.org, 2017.

Nicole Sandra-Yaffa Dumont and Chris Eliasmith. Accurate representation for spatial cognition using grid cells. In *42nd Annual Meeting of the Cognitive Science Society*, pages 2367–2373, Toronto, ON, 2020. Cognitive Science Society.

Chris Eliasmith. *How to Build a Brain: A Neural Architecture for Biological Cognition*. Oxford University Press, 2013.

Chris Eliasmith, Terrence C. Stewart, Xuan Choo, Trevor Bekolay, Travis DeWolf, Yichuan Tang, and Daniel Rasmussen. A large-scale model of the functioning brain. *Science*, 338 (6111):1202–1205, 2012.

Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Joshua B. Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2019. Curran Associates Inc.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '11, pages 317–330, New York, NY, USA, 2011. Association for Computing Machinery.

Isaac Joffe and Chris Eliasmith. Vector symbolic algebras for the Abstraction and Reasoning Corpus. https://arxiv.org/abs/2511.08747, 2025.

Aysja Johnson, Wai Keen Vong, Brenden M. Lake, and Todd M. Gureckis. Fast and flexible: Human program induction in abstract reasoning tasks. In *43rd Annual Meeting of the Cognitive Science Society*. Cognitive Science Society, 2021.

Ian T. Jolliffe. *Principal Component Analysis*. Springer New York, NY, 2002.

Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus, and Giroux, 2011.

Denis Kleyko, Dmitri A. Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, Part I: Models and data transformations. *ACM Computing Surveys*, 55(6), 2022.

Denis Kleyko, Dmitri Rachkovskij, Evgeny Osipov, and Abbas Rahimi. A survey on hyperdimensional computing aka vector symbolic architectures, Part II: Applications, cognitive models, and challenges. *ACM Computing Surveys*, 55(9), 2023.

Brent Komer, Terrence C. Stewart, Aaron R. Voelker, and Chris Eliasmith. A neural representation of continuous space using fractional binding. In *41st Annual Meeting of the Cognitive Science Society*, Montreal, QC, 2019. Cognitive Science Society.

Yann LeCun, Yoshua Bengio, and Geoffrey E. Hinton. Deep learning. *Nature*, 521:436–444, 2015.

Solim LeGris, Wai Keen Vong, Brenden M. Lake, and Todd M. Gureckis. A comprehensive behavioral dataset for the Abstraction and Reasoning Corpus. *Scientific Data*, 12(1380), 2025.

Aditya Menon, Omer Tamuz, Sumit Gulwani, Butler Lampson, and Adam Kalai. A machine learning framework for programming by example. In Sanjoy Dasgupta and David McAllester, editors, *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pages 187–195, Atlanta, Georgia, USA, 2013. PMLR.

Vijayaraghavan Murali, Letao Qi, Swarat Chaudhuri, and Chris Jermaine. Neural sketch learning for conditional program generation. In *International Conference on Learning Representations*, 2018.

Maxwell Nye, Luke Hewitt, Joshua Tenenbaum, and Armando Solar-Lezama. Learning to infer program sketches. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 4861–4870. PMLR, 2019.

Tony Plate. Holographic reduced representations: Convolution algebra for compositional distributed representations. In *Proceedings of the 12th International Joint Conference on Artificial Intelligence - Volume 1*, IJCAI'91, pages 30–35, San Francisco, CA, USA, 1991. Morgan Kaufmann Publishers Inc.

Tony A. Plate. *Holographic Reduced Representation: Distributed Representation for Cognitive Structures*. CSLI Publications, 2003.

Daniel Rasmussen and Chris Eliasmith. A neural model of rule generation in inductive reasoning. *Topics in Cognitive Science*, 3:140–153, 2011.

J. C. Raven. *The Performances of Related Individuals in Tests Mainly Educative and Mainly Reproductive Mental Tests Used in Genetic Studies*. University of London (King's College), 1936.

Jürgen Schmidhuber. Deep learning in neural networks: An overview. *Neural Networks*, 61:85–117, 2015.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2015.

Peter C. Wason and Jonathan St B. T. Evans. Dual processes in reasoning? *Cognition*, 3 (2):141–154, 1974.

Appendix A. Additional ARC-AGI Examples

In this appendix, we provide two more example ARC-AGI tasks (Figures 8 and 9) together with our solver's solutions to those tasks (Figures 10 and 11) to further illustrate the breadth of ARC-AGI and the capabilities of our solver.

Appendix B. DSL Details

In this appendix, we outline our DSL, delineating each of its object hypothesis primitives (Table 1), data types (Table 2), and transformation primitives (Tables 3 and 4).

Appendix C. VSA Representation Details

In this appendix, we explicitly define the VSA-based encoding schemes that we use to generate the perceptual representations that the NNs accept as input and describe how we generate the VSA-based primitive representations that the NNs produce as output.

As a baseline, we use a naive scheme that encodes the position and colour of each pixel in a grid G of size $r \times c$ as

$$\mathtt{G}_{\text{Naive}} = \textsc{EncodeGrid}_{\text{Naive}}(G) = \sum_{i=1}^{r}\sum_{j=1}^{c} \textsc{EncodeColour}(G_{ij}) \circledast \phi_{\mathbb{G}}(\textsc{Position}(i,j)) \tag{2}$$

where $\textsc{EncodeColour}(G_{ij})$ returns a representation of the colour of the pixel G_{ij} (i.e., `BLACK` if G_{ij} is black, `BLUE` if G_{ij} is blue, and so on), $\textsc{Position}(i,j)$ converts the coordinate pair (i,j) into a position vector relative to the centre of G, and $\phi_{\mathbb{G}}(\cdot)$ encodes that position vector as an SSP. This scheme can theoretically be used to detect any primitive because it encodes all information about G; however, detecting all primitives with this scheme may be difficult because the required information about G may not be encoded in a conducive manner.

Our first scheme encodes the positions of all background (i.e., black) and foreground (i.e., non-black) pixels in a grid G of size $r \times c$ as

$$\mathtt{G}_1 = \textsc{EncodeGrid}_1(G) = \sum_{i=1}^{r}\sum_{j=1}^{c} \textsc{EncodeBackFore}(G_{ij}) \circledast \phi_{\mathbb{G}}(\textsc{Position}(i,j)) \tag{3}$$

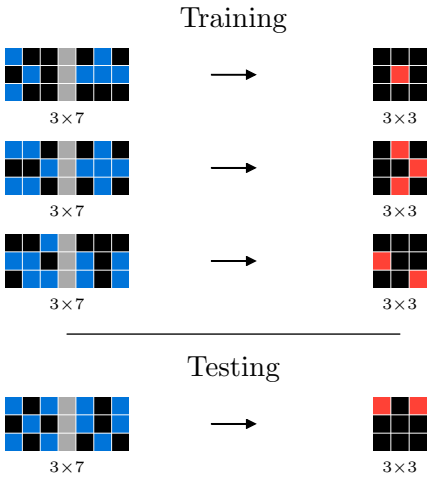


Figure 8: ARC-AGI task `0520fde7`.

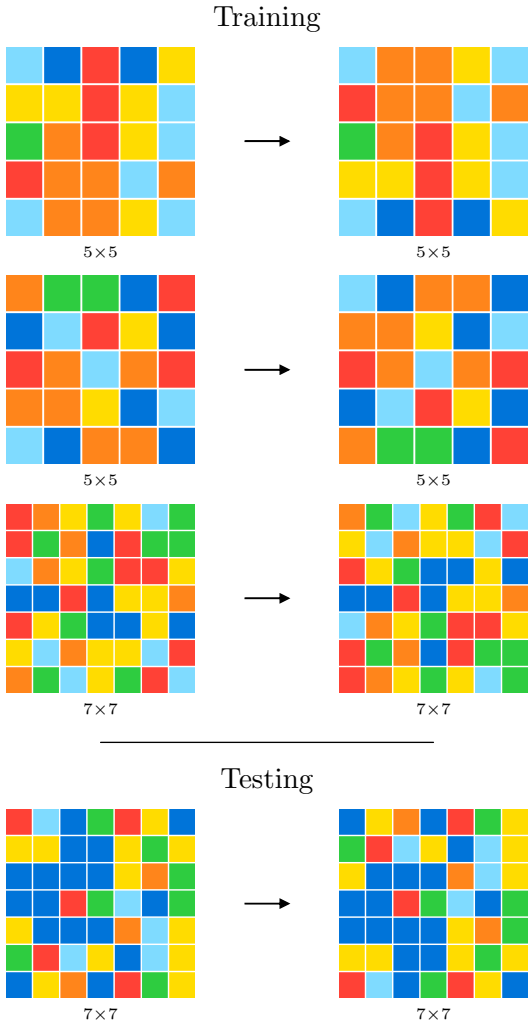


Figure 9: ARC-AGI task `68b16354`.

1. $\mathcal{O}_\mathrm{I} \leftarrow \textsc{PartitionHypothesis}(G_\mathrm{I})$
2. $\mathcal{O}_\mathrm{O} \leftarrow \textsc{Recolour}($
   $\textsc{PixelwiseAnd}(\mathcal{O}_\mathrm{I}),$
   $\textsc{ColourConstant}(\mathrm{Red}))$
3. $G_\mathrm{O} \leftarrow \textsc{DecodeObjects}(\mathcal{O}_\mathrm{O})$

Figure 10: Our solver's solution program for ARC-AGI task `0520fde7`.

1. $\mathcal{O}_\mathrm{I} \leftarrow \textsc{GridHypothesis}(G_\mathrm{I})$
2. $\mathcal{O}_\mathrm{O} \leftarrow \textsc{VerticalFlip}(\mathcal{O}_\mathrm{I})$
3. $G_\mathrm{O} \leftarrow \textsc{DecodeObjects}(\mathcal{O}_\mathrm{O})$

Figure 11: Our solver's solution program for ARC-AGI task `68b16354`.

Table 1: All object hypothesis primitives in our DSL.

Primitive	Description
GridHypothesis	The whole grid forms one object.
PartitionHypothesis	Each regional division of the grid that contains only one colour forms one object.
ColourHypothesis	Each group of pixels of the same colour, regardless of spatial contiguity, forms one object.
TwoConnectedHypothesis	Each group of two-connected pixels (i.e., pixels sharing an edge or corner) of the same colour forms one object.
OneConnectedHypothesis	Each group of one-connected pixels (i.e., pixels sharing an edge) of the same colour forms one object.

Table 2: All data types in our DSL.

Type	Description
Object	Stores a grid mask containing the colour and position of all foreground pixels. All DSL primitives are concerned with this type.
Colour	Stores one of the 10 possible colours in ARC-AGI grids (i.e., either black, blue, red, green, yellow, grey, pink, orange, cyan, or purple).
Number	Stores a natural number.
Boolean	Stores a binary truth value (i.e., either `True` or `False`).

Table 3: All Object-producing transformation primitives in our DSL.

Primitive	Parameters	Description
ArgMax	Object, Number	Selects the object or objects with the largest corresponding numerical values.
ArgMin	Object, Number	Selects the object or objects with the smallest corresponding numerical values.
Crop	Object	Crops each object's mask around its foreground pixels.
Encircle	Object, Colour	Surrounds the foreground pixels of each object with the provided colour.
FillHoles	Object, Colour	Converts the enclosed background pixels of each object into foreground pixels of the provided colour.
FillLines	Object, Colour	Converts linearly bounded background pixels into foreground pixels of the provided colour.
Filter	Object, Boolean	Selects the objects with corresponding `True` boolean values.
HorizontalFlip	Object	Reflects each object about the grid's vertical axis.
HorizontalStack	Object, Object	Concatenates two objects horizontally.
Hollow	Object	Converts the enclosed foreground pixels of each object into background pixels.
PixelwiseAnd	Object	Overlays and retains pixels common to all objects.
PixelwiseNot	Object	Interchanges all background and foreground pixels.
PixelwiseOr	Object	Overlays and retains pixels present in any object.
PixelwiseXor	Object	Overlays and retains pixels present in one object.
Recolour	Object, Colour	Changes all foreground pixels of each object to the provided colour.
Rectangify	Object, Colour	Converts rectangularly bounded background pixels into foreground pixels of the provided colour.
Rotate90	Object	Rotates each object clockwise by 90 degrees.
Rotate180	Object	Rotates each object by 180 degrees.
Rotate270	Object	Rotates each object clockwise by 270 degrees.
Scale	Object, Number	Upscales each object by the provided numerical value.
Transpose	Object	Transposes each object about the grid's diagonal.
VerticalFlip	Object	Reflects each object about the grid's horizontal axis.
VerticalStack	Object, Object	Concatenates two objects vertically.
Xify	Object, Colour	Draws a maximal $\times$ of the provided colour centred at each pixel of each object.

Table 4: All other transformation primitives in our DSL.

Primitive	Parameters	Description
ColourConstant	—	Creates a constant colour.
GetColour	Object	Gets the majority colour of each object.
NumberConstant	—	Creates a constant number.
GetColourCount	Object	Counts the number of distinct colours in each object.
GetHeight	Object	Measures the maximum height of each object's foreground pixels.
GetPixelCount	Object	Counts the total number of foreground pixels in each object.
GetWidth	Object	Measures the maximum width of each object's foreground pixels.
HasHoles	Object	Determines whether each object's foreground pixels fully enclose any background pixels.
HasHorizontal-Symmetry	Object	Determines whether each object's foreground pixels are symmetrical about its vertical axis.
HasVertical-Symmetry	Object	Determines whether each object's foreground pixels are symmetrical about its horizontal axis.

where $\textsc{EncodeBackFore}(G_{ij})$ returns a representation of the status of the pixel G_{ij} (i.e., a representation `BACKGROUND` of the background if G_{ij} is black, and a representation `FOREGROUND` of the foreground otherwise), and $\textsc{Position}(\cdot)$ and $\phi_{\mathbb{G}}(\cdot)$ are as before. This scheme is especially useful for detecting primitives that affect grid size (e.g., Crop) or interchange background and foreground pixels (e.g., Encircle), as well as in cases where the colours of the foreground pixels are completely irrelevant.

Our second scheme encodes the position and colour of all foreground pixels in a grid G of size $r \times c$ as

$$\mathtt{G}_2 = \textsc{EncodeGrid}_2(G) = \sum_{i=1}^{r} \sum_{j=1}^{c} \textsc{EncodeForeColour}(G_{ij}) \circledast \phi_{\mathbb{G}}(\textsc{Position}(i, j)) \quad (4)$$

where $\textsc{EncodeForeColour}(G_{ij})$ returns a representation of the colour of the pixel G_{ij} only if G_{ij} is a foreground pixel (i.e., the zero vector if G_{ij} is black, but `BLUE` if G_{ij} is blue, `RED` if G_{ij} is red, and so on), and $\textsc{Position}(\cdot)$ and $\phi_{\mathbb{G}}(\cdot)$ are as before. This scheme is especially useful for detecting primitives that re-permute pixels of different colours (e.g., VerticalFlip), as well as in cases where the background pixels are irrelevant.

Our third scheme encodes the number of pixels of each foreground colour in a grid G that contains pixels of the colours $\mathcal{C}$ as

$$\mathtt{G}_3 = \textsc{EncodeGrid}_3(G) = \sum_{C \in \mathcal{C}} \textsc{EncodeForeColour}(C) \circledast \phi_{\mathbb{R}}(\textsc{Count}(C, G)) \quad (5)$$

where EncodeForeColour$(\cdot)$ is as before, Count(C, G) returns the number of pixels of the colour C in G, and $\phi_{\mathbb{R}}(\cdot)$ encodes that natural number as an SSP. This scheme is especially useful for detecting primitives that add colours to (e.g., Encircle), remove colours from (e.g., Filter), or change the colours in (e.g., Recolour) G, as well as in cases where the positions of the foreground pixels are irrelevant.

As a baseline, we use human-engineered embeddings as VSA primitive representations. We generate our human-engineered representations as follows: first, we manually assign each primitive a two-dimensional feature vector that clusters related primitives together; and, second, we encode these two-dimensional feature vectors as SSPs.

Our learned VSA primitive representations come from the embeddings learned by f_{Naive}. We generate our learned representations as follows: first, we extract the final weight layer $\mathbf{W} \in \mathbb{R}^{K \times M}$ of f_{Naive}; second, we reduce the learned embeddings of the K primitives from M dimensions to two-dimensional vectors using linear principal component analysis (Jolliffe, 2002); third, we encode these two-dimensional feature vectors as SSPs; and, fourth, we tag each resulting SSP with the type that its corresponding primitive produces by binding it with a discrete type representation.

Appendix D. Hyperparameter Details

In this appendix, we provide the values of the hyperparameters that we use in our experiments.

Our VSA perceptual representations use $N = 1024$ dimensions, and our VSA program representations use $N = 4096$ dimensions. As a result, f_{Naive} has 2048 input neurons in addition to its K output neurons, and f_{VSA} outputs vectors of 4096 dimensions.

As previously explained, f_{Naive} is implemented as a simple multi-layer perceptron with logistic output activations and trained by mini-batch gradient descent on binary cross-entropy loss. We instantiate f_{Naive} with two hidden layers of 512 and 128 neurons; as a result, f_{Naive} comprises 1.1 million trainable parameters. We train f_{Naive} for 1000 epochs with early stopping based on increasing validation loss, a batch size of 256, and a learning rate of 0.0001. We strongly regularize f_{Naive} using an L_2 penalty on the weights with a coefficient of 1, dropout (Srivastava et al., 2014) with a rate of 0.5, and label smoothing (Szegedy et al., 2015) with a coefficient of 0.01. Our training dataset has 280,000 samples and was generated so that each of our DSL's primitives appears in at least 10,000 samples.

Appendix E. Additional Results

In this appendix, we list all the ARC-AGI tasks solved by our solver, including tasks from both ARC-AGI-1-Train and ARC-AGI-1-Eval, together with the median number of programs that our best solver proposed before solving them (Table 5).

Additionally, we conduct all experiments again with NNs trained on 10× fewer data (i.e., a training dataset of 28,000 samples generated so that each of our DSL's primitives appears in at least 1,000 samples). Performance is worse, but all the same patterns emerge (Figures 12, 13 and 14), indicating that our solver is robust to changes in data quantity.

Note that one of the 54 ARC-AGI-1-Train tasks solved by our solver, ARC-AGI task `b1948b0a`, was omitted from the task subset used in the task-solving efficiency experi-

Table 5: All the ARC-AGI tasks solved by our solver (54 tasks from ARC-AGI-1-Train and 14 tasks from ARC-AGI-1-Eval, as separated by the rule) together with the median number of programs that our best solver proposed before solving each.

Task ID	**Eff.**	**Task ID**	**Eff.**	**Task ID**	**Eff.**	**Task ID**	**Eff.**
00d62c1b	1	4c4377d9	5	99b1bc43	4	e98196ab	1
0520fde7	2	50cb2852	13	9dfd6313	1	ed36ccf7	1
0b148d64	1553	56ff96f3	7	a416b8f3	1	f25fbde4	8
1cf80156	1	60b61512	1	a5313dff	5	195ba7dc	2
1f85a75f	32	6150a2bd	1	a87f7484	34	31d5ba1a	4
22168020	10	623ea044	1	ac0a08a4	1	34b99a2b	2
22eb0ac0	2	67a3c6ac	1	b1948b0a	244	506d28a5	2
23b5c85d	26	68b16354	1	b91ae062	1	5d2a5c43	2
253bf280	1	6d0aefbc	3	bb43febb	69	60c09cac	1
25d8a9c8	78	6d75e8bb	1	be94b721	42	84db8fc4	107
28bf18c6	8	6f8cd79b	56	c59eb873	1	aa18de87	1
31aa019c	26	6fa7a44f	6	c9e6f938	16	bf89d739	37
3428a4f5	3	72ca375d	30	ce22a75a	16	d19f7514	3
3aa6fb7a	3	7468f01a	5	ce4f8723	2	d4b1c2b1	1
3c9b0459	1	74dd1130	1	dae9d2b5	3	d56f2372	41
4258a5f9	4	8be77c9e	6	dbc1a6ce	1	e133d23d	2
4347f46a	1	9172f3a0	3	ded97339	1	e7639916	26

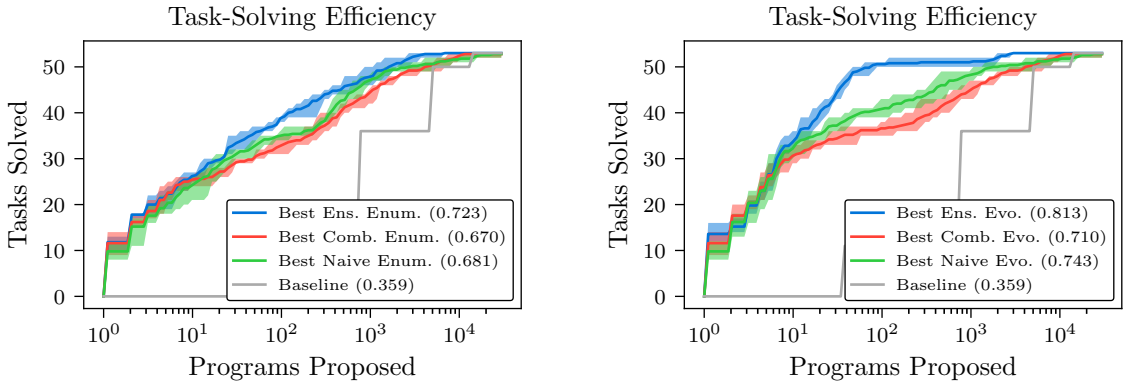


Figure 12: As before (see Figure 5), our solver performs better when it uses an NN ensemble for scoring, regardless of whether it uses enumeration (left) or evolution (right) for search. Also as before, lines are means and shaded regions cover the minimum and maximum number of tasks solved across five trials; numbers are normalized areas under the mean curves.

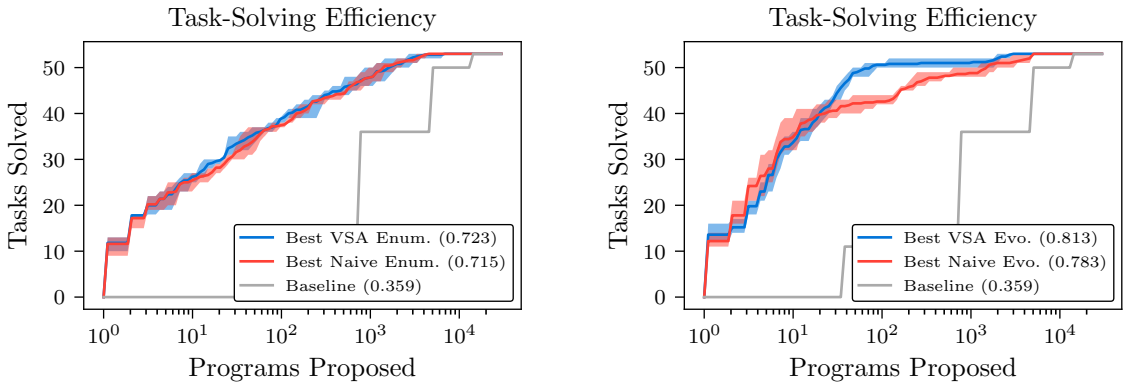


Figure 13: As before (see Figure 6), our solver performs slightly better when it uses VSA representations for scoring, regardless of whether it uses enumeration (left) or evolution (right) for search.

ments because our solver's synthesized solution program for that task, which relied on implementation-specific quirks of our DSL, was unsatisfactory. Note also that the values of the constants in a proposed program that invokes constant-producing primitives (i.e., ColourConstant, NumberConstant) are determined by naive enumeration for simplicity, and each variant (i.e., constant assignment) of that program is not counted as another proposed program.

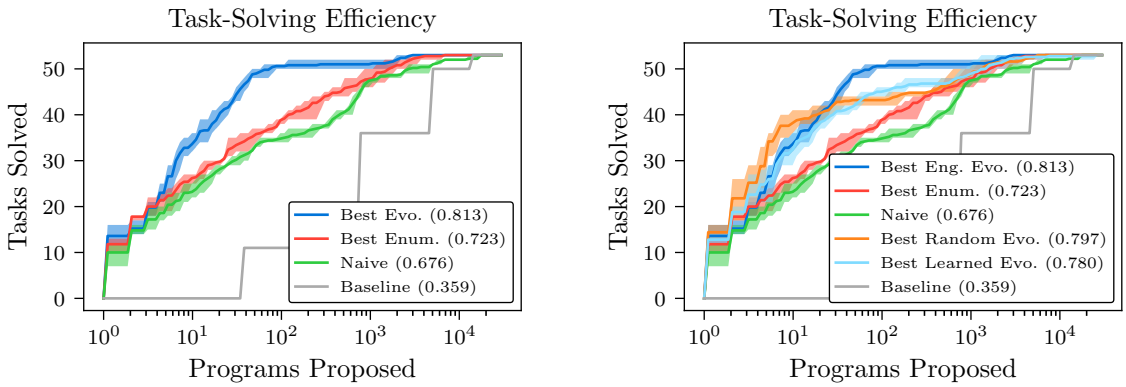


Figure 14: As before (see Figure 7), our solver performs significantly better when it uses VSA-powered evolution for search (left), even with random or learned primitive representations (right).